

Stochastic Simulation Assignment 2

Module 8

Tycho Borm Mark Tijhaar

May 27, 2026

1 Introduction

We are asked to simulate a limit order book with market impact. This means that whenever trades happen, the mid price updates correspondingly. So for instance, if an incoming buy market or limit order triggers a trade, then the price should go up, since people are apparently willing to buy at the current price. We will measure statistics about this process using the batch and regenerative means methods.

Orders arrive according to a Poisson process where rate

$$\lambda(m_t) = \lambda_0 \exp\left(-\alpha(m_t - m^*)^2\right) + \lambda_{\min}, \quad m^* = 100$$

depends on the current mid price m_t . Here $\lambda_{\min}$ is the minimum arrival rate, and $\lambda_0 + \lambda_{\min}$ is the maximum arrival rate

Whenever the arrival rate changes, this of course also impacts when the next order should arrive. So the next arrival should be rescheduled.

The chance for any incoming order to be a limit order is 0.7, otherwise it will be a market order. Both types of orders have a 50-50 chance to be either buy (bid) or sell (ask). Whenever a limit order is added to the book it checks if that immediately triggers a trade, a trade is sufficient and therefore will be made if best bid $\geq$ best ask. If there is no immediate sufficient trade for the order, it will be saved in a list, in case later on there will be. Market orders immediately take the best possible trade they can make at that moment

All limit orders will get cancelled after an exponentially distributed random wait time, with a mean of 60.

Whenever the price is far away from the mean price m^* , then mean reversion activates taking the price closer to the mid.

$$m_t^+ = m_t + \eta v - \kappa(m_t - m^*).$$

Otherwise the midprice updates as

$$m_t^+ = m_t + \eta v.$$

Here m_t^+ is the new mid price, m_t is the old mid price, m^* is the equilibrium price, v is the trade volume, η is the market impact and κ is the intensity of the mean reversion

Our code can be found in the github repository in the file `LimitOrderBookWithMarketImpact.py`

2 Method

To run this simulation we use a total of 6 different classes, of which 3 are Events. Firstly, we use a Statistics class, which keeps track of all statistics for one batch. We also use a class to represent limit orders, so each one can keep track of their price, if it is a bid or ask order and at what time they arrived. The last non-event class we use is the Book class. This class keeps track of all information we need to carry out our simulation. This is because we can't give any parameters, like the statistics, to our simulation, so we bundle all of them in one class so all events only need one passed parameter, which is the book. The book all of the following parameters:

- λ , the current arrival rate
- λ_0 , the maximum arrival rate
- $\lambda_{\min}$, the minimal arrival rate
- α , the rate at which arrival rate decays away from m^*

- q_L , the chance an arriving order is a limit order
- $\bar{v}$, the average trade size
- δ_{bar} , how far a trades price is placed from the current price on average
- m^* , the equilibrium price
- $\bar{\tau}$, the mean waiting time before limit orders cancel
- Δ_{max} how far the current mid price has to be from the equilibrium before mean reversal activates
- κ mean reversal strength
- η is the impact coefficient, describing how much the market price fluctuates after a trade

Besides these parameters, the book also keeps track of all existing limit orders and handles any interaction between limit orders, or between limit orders and market orders. The book is also responsible for logging any statistics. It also keeps track of the next OrderArrival, because whenever a trade happens and the arrival rate changes, the OrderArrival needs to be rescheduled according to the new arrival rate. This happens by just sampling a new exponential random variable with the new arrival rate, and then scheduling the arrival from the current time. This works because the exponential distributions are memoryless, so if the next event hasn't arrived yet and the rate changes, this is exactly equivalent to just sampling a new value. Lastly the book also keeps track of all the settings for the batch means and regenerative means methods.

Then for our event classes, we have the SampleMarketVolatility class, which, as the name suggests, samples the market price when activated, and then schedules another sample event 1 unit from now. This is because this sampling is done independently from the rest of the simulation at 1 time unit intervals. The volatility for a batch is equal to the standard deviation of the log-returns of all intervals $[t + \Delta t)$, which are equal to

$$\ln \left(\frac{m_{t+\Delta t}}{m_t} \right)$$

The last two event classes are the OrderArrival and CancelEvent classes. Whenever an OrderArrival executes, it generates a new market or limit order, handles it and if it is a limit order, schedules its corresponding cancel event, then lastly it schedules a new OrderArrival according to the current arrival rate.

When the CancelEvent fires, all it does is remove the linked order from the book and log the cancellation in the statistics.

At initialization, a simulation and book object are made and a first Object arrival is scheduled at $t=0$, after which the simulation is described by Algorithm 1 with parameters

$\lambda_0 = 5$, $\lambda_{min} = 0.5$, $\alpha = 0.05$, $q_L = 0.7$, $\bar{v} = 10$, $\bar{\delta} = 0.5$, $\bar{\tau} = 60$, $m^* = 100$, $\Delta_{max} = 5$, $\kappa = 0.1$,

$(\eta = 0.02)$ $(\eta = 0.10)$.

Algorithm 1 LimitOrderBookWithMarketImpact simulation

```
while Simulation queue not empty and simulation not terminated do
   $E \leftarrow$  next event scheduled
  if  $E$  is OrderArrival then
    Schedule next OrderArrival
    Generate a new order and handle it according to Algorithm 2
  else if  $E$  is CancelEvent then
    Remove the corresponding order from the queue
    Update spread statistics
    Update cancel statistics
  else if  $E$  is SampleMarketVolatility then
    Update volatility stats
  end if
end while
Calculate cancel rate
Calculate spread statistic
Return all statistics
```

Algorithm 2 Generating & handling new limit order

```
if Order is a limit order with chance  $q_L$  then
  Determine whether order is bid or ask limit order & generate price
  if the price is unacceptable ( $\leq 0$ ) then
    Return
  end if
  Add it to the corresponding queue in the book
  if Its price is compatible with the best order in the other queue then
    Run Algorithm 3 with these two orders
  end if
else if Order is a market order then
  Determine whether order is buy or sell market order
  if Opposing queue is not empty then
    Run Algorithm 3 with the best price out of the opposing queue with arrival time as a
    tie breaker
  else
    Update reject statistics
  end if
end if
Calculate cancel rate
Calculate spread statistic
Return all statistics
```

Algorithm 3 Process Trade

Either 1 or 2 orders are passed depending on if the trade came from a market or limit order
 Remove all passed orders from their queues
 Generate volume for the order
 Recalculate the mid price and arrival rate
 Reschedule the next OrderArrival according to the new arrival rate
 Update time to fill stats
 update arrival rate stats
 update bid-ask spread stats

3 Verification cases

3.1 V1

For the first verification case, we set $\alpha = 0$ and $\eta = 0$. This means that the arrival rate can never change. We then set the arrival rate to $\lambda + \mu$ where λ should denote the arrival rate of limit orders, and μ the rate of market orders. Then we set the chance of a limit order to $\frac{\lambda}{\lambda + \mu}$, so that the arrival rates are realised.

Then we set every limit order to a bid order of a fixed value, so the book queue becomes FIFO, and every market order to a sell order. Since the arrival rate of market orders is memoryless, this is exactly the same as scheduling a market order arriving after each limit order reaches the head of the queue, so this reduction should model an M/M/1 queue system. This should mean the average waiting time and average queue waiting time should follow those of an M/M/1 queue:

$$\mathbb{E}[W] = \frac{1}{\mu - \lambda}, \quad \mathbb{E}[W_q] = \frac{\lambda}{\mu(\mu - \lambda)}.$$

Running the simulation with $\lambda = 0.5$ and $\mu = 1$, which results in a total order arrival rate of 1.5 per time unit, gives these result

$$\overline{W} = 1.98, \overline{W}_q = 0.975,$$

while with the given λ and μ they should be 2 and 1 respectively, so the simulation passed V1.

3.2 V2

The second verification case works the same as V1, except the market orders arrive exactly $\frac{1}{\mu}$ time units after a limit order reaches the head of the queue (so when it is the oldest order that is still in the book). To run this, we needed an extra event class that is scheduled $\frac{1}{\mu}$ time units after a limit order reaches this position. This transforms the system into an M/D/1 queue, which should have a queue waiting time of

$$\mathbb{E}[W_q] = \frac{\rho}{2\mu(1 - \rho)}, \quad \rho = \frac{\lambda}{\mu}.$$

With the same lambda and mu parameters, the simulation calculates $\overline{W}_q$ to be 0.501, while the value should approach 0.5, so the simulation also passed V2.

4 Data collection

For the data collection we used both the batch means and regenerative methods. For the batch means we chose a warm-up period of 180 time units, since 95% of all cancellation events will

occur within that time. And then the queues should in theory have a nice distribution of orders with different lifespans. Our batch lengths are a standard 4 times as large.

Our regenerative state was an empty book, so both the bid and ask queues are empty, with a mid price less than 2.5 away from the equilibrium. We chose this number because in this simulation, prices can be any float, so you never have a guarantee of actually ever hitting the equilibrium, and any smaller error for the regenerative state did not happen often enough in practice for the simulation time to not be unreasonably long.

We ran the batch method for different amounts of batches to estimate the number of batches we need according to

$$r \geq \frac{t_{r-1, 1-\alpha/2}^2 S^2(r)}{\left(\frac{\delta}{1+\delta} \hat{X}\right)^2}$$

Which held true for the amount of batches greater than about 50, so this is the amount of batches and cycles we are going to use.

For actually gathering the statistics apart from the before mentioned volatility, which has it's own event to fire every 1 unit, we used a wrapper class to keep track of all statistics for 1 batch/cycle and saved that class in a list to calculate all statistics and confidence intervals after the simulation.

The other statistics we keep track of are the time-averaged bid-ask spread, where all intervals where any of the two queues is empty is skipped. The statistic is updated after any Order is added or removed from any queue. We also keep track of the cancellation rate, which counts the amount of cancellations whenever one happened, and then divide by the total runtime of that batch to get the amount of cancellations per second. Similarly for the market order rejections we count whenever the opposing queue is empty and a market order is rejected, and at the end of the batch we divide by the batch length.

We keep track of the average time to fill by logging the difference between the current time and the arrival time of any order when it is filled, and take the average for every batch.

We also keep track of the arrival rate whenever it changes. The plot of the arrival rate over time is only for one batch, because the plot for the entire run looks the same, but is just longer so the data is less visible.

5 Results

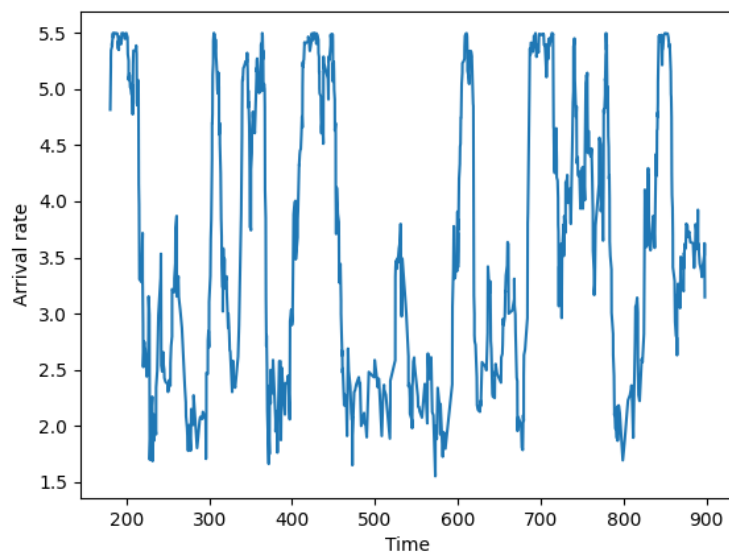


Figure 1: fig:Arrival rate over time for one batch with $\eta = 0.02$

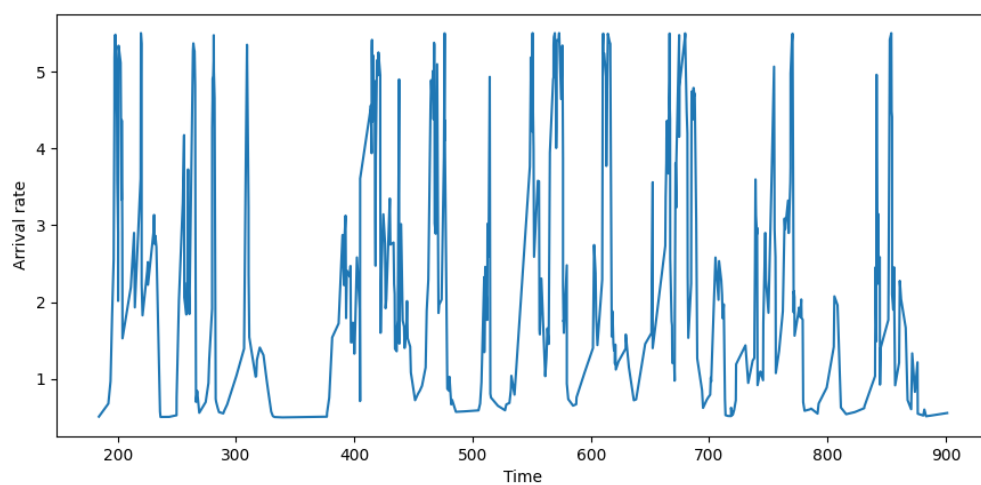


Figure 2: Arrival rate over time for one batch with $\eta = 0.1$

Metric	Value	95% CI
Time-averaged bid-ask spread	0.08	(0.05, 0.12)
Average time-to-fill	8.94	(8.70, 9.19)
Cancellation rate	0.52	(0.50, 0.53)
Rejection rate	0.011	(0.007, 0.014)
Average volatility	0.0044	

Table 1: Metrics with batch method and $\eta = 0.02$

Metric	Value	95% CI
Time-averaged bid-ask spread	0.19	(0.08, 0.29)
Average time-to-fill	6.85	(6.65, 7.06)
Cancellation rate	0.174	(0.167, 0.181)
Rejection rate	0.049	(0.044, 0.054)
Average volatility	0.02	

Table 2: Metrics with batch method and $\eta = 0.1$

Metric	Value	95% CI
Time-averaged bid-ask spread	0.09	(0.04, 0.14)
Average time-to-fill	7.00	(6.00, 7.99)
Cancellation rate	0.40	(0.34, 0.46)
Rejection rate	0.16	(0.00, 0.33)
Average volatility	0.0045	

Table 3: Metrics with regenerative method and $\eta = 0.02$

Metric	Value	95% CI
Time-averaged bid-ask spread	0.24	(0.09, 0.39)
Average time-to-fill	5.60	(4.82, 6.39)
Cancellation rate	0.14	(0.12, 0.16)
Rejection rate	0.44	(0.02, 0.85)
Average volatility	0.021	

Table 4: Metrics with regenerative method and $\eta = 0.1$

Because the arrival rate is described by

$$\lambda(m_t) = \lambda_0 \exp\left(-\alpha(m_t - m^*)^2\right) + \lambda_{\min}, \quad m^* = 100,$$

the graph for the higher η is lower on average, because bigger jumps in price mean the mid price is going to be further away from the equilibrium on average. This is also why the volatility is higher for the higher η . Since volatility is based on how big the jumps in mid price are, which are larger with a larger η .

The bid-ask spread is higher for a higher η , this is also explainable, since the mid price jumps way more, the prices of limit orders are also going to be more sporadic and thus the bid-ask spread might be lifted up a little bit.

This jumping also means the orders will get accepted, quicker though, because there is less chance the simulation will stay in the same price range for a long time which means someone will over/under bid quickly.

Lower time to fill also means orders will get cancelled less often, since they will be filled before they are cancelled. But with more orders filled and a lower arrival rate due to sporadic mid prices, there will be less orders in the queue, which means that market orders will find empty queues more often, leading to a higher rejection rate.